

Compressió de dades i imatges - GEI - FIB

Tema 2: Codis de Huffman

Anna de Mier, Lluís Vena
anna.de.mier@upc.edu, lluis.vena@upc.edu

Departament de Matemàtiques • Universitat Politècnica de Catalunya

Febrer 2026

Capítol 3 de [Introduction to Data Compression](#) **Sayood, Khalid** Morgan
Kaufmann, 2012, 4a ed.

- 1 Algorisme
- 2 Teorema de la codificació sense soroll (Primer teorema de Shannon)
- 3 Variants
 - Codis adaptatius
 - Codis de Shannon-Fao

Algorisme

Recordem:

Una font $\mathcal{S}$ emet símbols amb probabilitats $\mathcal{P}(s_i) = p_i$. La longitud mitjana d'una codificació $\mathcal{C}$ que codifica s_i amb c_i és $\tilde{l}(\mathcal{C}) = \sum l(c_i)p_i$.

L'algorisme de codificació de Huffman es basa en les observacions següents per minimitzar $\tilde{l}(C)$:

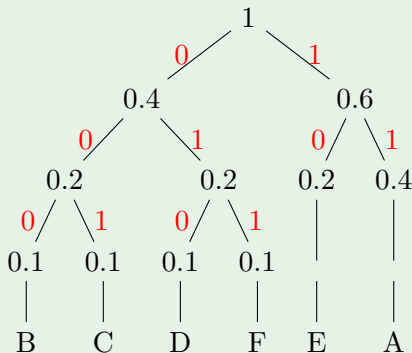
- Els símbols que apareixen més sovint s'han de codificar amb paraules més curtes
- Els dos símbols menys freqüents es codifiquen amb paraules de la mateixa longitud

A més, imposen que els dos símbols menys freqüents es codifiquin amb c_0 i c_1

Algorisme

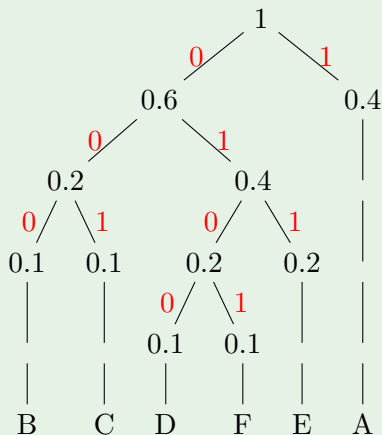
Exemple

$(A, 0.4), (B, 0.1), (C, 0.1), (D, 0.1), (E, 0.2), (F, 0.1)$



$A \leftrightarrow 11, B \leftrightarrow 000, C \leftrightarrow 001, D \leftrightarrow 010, E \leftrightarrow 10, F \leftrightarrow 011 \quad \tilde{l} = 2.4 \text{ bits/símbol.}$

Una altra possibilitat:



$A \leftrightarrow 1$, $B \leftrightarrow 000$, $C \leftrightarrow 001$, $D \leftrightarrow 0100$, $E \leftrightarrow 011$, $F \leftrightarrow 0101$ $\tilde{l} = 2.4$ bits/símb.

Observacions:

- i) Els codis de Huffman no són únics. (Veurem que tots tenen la mateixa longitud mitjana, però les variàncies poden ser diferents.)
- ii) Per construcció, són prefixos.
- iii) Si $q > 2$ en la primera iteració s'han d'agrupar t símbols complint:
 $2 \leq t \leq q$, $t \equiv n \pmod{q-1}$.
Això es deu a que un arbre q -ari amb m vèrtexs interns té $(q-1)m + 1$ fulles
- iv) Les longituds de les t paraules més llargues són iguals.
- v) Les t paraules més llargues difereixen en l'última lletra.

Teorema

Els codis de Huffman tenen longitud mitjana mínima.

Demostració

Observacions prèvies:

Suposem $p_1 \geq p_2 \geq \dots \geq p_n$.

Sigui $\mathcal{C}_H$ codi de Huffman amb paraules de longituds $l_1 \leq l_2 \leq \dots \leq l_{n-1} = l_n$. Sigui $\mathcal{C} = \{c_1, \dots, c_n\}$ codi amb paraules de longituds $m_1, m_2, \dots, m_n$. Podem suposar que $m_1 \leq m_2 \leq \dots \leq m_{n-1} = m_n$ ja que:

- Si les longituds no estan ordenades, reordenant-les tenim un codi amb $\tilde{l}$ menor,
- Si $m_{n-1} < m_n$ podem eliminar lletres de c_n fins que tingui la mateixa longitud que c_{n-1} (prefix) resultant un codi amb $\tilde{l}$ menor.

Continuació demostració

$\tilde{l}(\mathcal{C}_H) \leq \tilde{l}(\mathcal{C})$ per inducció sobre n , nombre de paraules del codi:

❶ $n = 1, \tilde{l}(\mathcal{C}_H) = 1 \leq \tilde{l}(\mathcal{C}) \checkmark$

❷ Cert fins a $n - 1 \Rightarrow$ cert per a n . Sigui $\mathcal{C}_H$ codi de Huffman per a la font $\mathcal{S}$:
 $p_1 \geq p_2 \geq \dots \geq p_n$

Sigui $\bar{\mathcal{C}}_H$ codi de Huffman per a la font $\bar{\mathcal{S}}$: $p_1 \geq p_2 \geq \dots \geq p_{n-2}, p_{n-1} + p_n$
Aleshores:

$$\tilde{l}(\mathcal{C}_H) = p_1 l_1 + \dots + p_{n-1} l_{n-1} + p_n l_n \stackrel{l_{n-1}=l_n}{=} p_1 l_1 + \dots + (p_{n-1} + p_n) l_n$$

$$\tilde{l}(\bar{\mathcal{C}}_H) = p_1 l_1 + \dots + p_{n-2} l_{n-2} + (p_{n-1} + p_n) [l_n - 1] \quad (\text{per construc. de Huffman}).$$

$$\text{Comparant } \tilde{l}(\bar{\mathcal{C}}_H) = \tilde{l}(\mathcal{C}_H) - (p_{n-1} + p_n).$$

Sigui ara $\mathcal{C}$ un codi per a la font $\mathcal{S}$. Si c_n i c_{n-1} no són fills del mateix node, sigui c_i el sibling de c_n ; podem intercanviar c_{n-1} i c_i sense canviar la longitud mitjana. Construïm $\bar{\mathcal{C}}$ un codi per a la font $\bar{\mathcal{S}}$ eliminant c_{n-1} i c_n i afegint $\bar{c}_{n-1}$ construïda suprimint l'última lletra de c_n (o de c_{n-1}).

$$\text{Aleshores } \tilde{l}(\bar{\mathcal{C}}) = \tilde{l}(\mathcal{C}) - (p_{n-1} + p_n)$$

$$\text{Per H.I. } \tilde{l}(\bar{\mathcal{C}}) \geq \tilde{l}(\bar{\mathcal{C}}_H), \text{ llavors } \tilde{l}(\mathcal{C}) - (p_{n-1} + p_n) \geq \tilde{l}(\mathcal{C}_H) - (p_{n-1} + p_n) \text{ i}$$

$$\tilde{l}(\mathcal{C}) \geq \tilde{l}(\mathcal{C}_H). \checkmark$$

Teorema

Els codis de Huffman tenen longitud mitjana mínima.

La variància de les longituds és $\text{Var}(\mathcal{C}) = \sum (l(c_i) - \tilde{l}(\mathcal{C}))^2 p_i$.

Observació

Per obtenir variància menor, quan hem de combinar les dues probabilitats més petites prioritzem sempre aquelles que siguin el menys “combinades” possible. Per exemple, si $p_n + p_{n-1} = 0.2$, $p_{n-2} = 0.15$, $p_{n-3} = 0.2$, combinem p_{n-2} i p_{n-3} en la propera iteració. (Compareu els arbres de les slides 4 i 5.)

Teorema de la codificació sense soroll

Lema (Gibbs)

Siguin:

$p_1, \dots, p_n$ reals tals que $p_i > 0$ amb $\sum p_i = 1$,

$x_1, \dots, x_n$ reals tals que $x_i > 0$ amb $\sum x_i \leq 1$. Aleshores:

$$\sum p_i \log \frac{1}{p_i} \leq \sum p_i \log \frac{1}{x_i}.$$

Proof.

Fixem p_i i definim $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $f(x_1, \dots, x_n) = \sum \left(p_i \log \frac{1}{x_i} - p_i \log \frac{1}{p_i} \right)$. Si busquem el mínim de f amb la condició $\sum x_i \leq 1$, $x_i > 0$, fàcilment (recordeu M2) obtenim que s'assoleix quan $x_i = p_i$. □

Primer teorema de Shannon – codificació sense soroll, V1

Sigui $\mathcal{C}_H$ un codi de Huffman per a una font $\mathcal{S}$, aleshores

$$H(\mathcal{S}) \leq \tilde{l}(\mathcal{C}_H) < H(\mathcal{S}) + 1.$$

Proof.

$p_1 \geq p_2 \geq \dots \geq p_n > 0$. $\mathcal{C}_H = \{c_1, \dots, c_n\}$ compleix la desigualtat de Kraft-McMillan, $\sum 2^{-l_i} \leq 1$.

- Primera desigualtat:

$$H(\mathcal{S}) = \sum p_i \log \frac{1}{p_i} \stackrel{\text{Gibbs}}{\leq} \sum p_i \log \frac{1}{2^{-l_i}} = \sum p_i l_i = \tilde{l}(\mathcal{C}_H)$$

- Segona desigualtat: Definim $a_i \equiv \lceil \log \frac{1}{p_i} \rceil \geq \log \frac{1}{p_i} > 0$ i.e. $a_i \geq 1$. A més $-a_i \leq \log p_i \Rightarrow 2^{-a_i} \leq p_i \Rightarrow \sum 2^{-a_i} \leq \sum p_i = 1$. Pel teorema de Kraft, existeix un codi $\mathcal{C}$ amb paraules de longituds $a_1, \dots, a_n$. Aleshores

$$\tilde{l}(\mathcal{C}_H) \leq \tilde{l}(\mathcal{C}) = \sum a_i p_i \stackrel{\text{def.}}{<} \sum p_i \left(\log \frac{1}{p_i} + 1 \right) = H(\mathcal{S}) + 1.$$

□

Exemple

$\mathcal{S} = \{(s_1, 1/4), (s_2, 3/4)\}$. Per a aquesta font $H(\mathcal{S}) = 0.811$.

- Si codifiquem cada símbol:

- ▶ $s_1 \rightarrow 0$,
- ▶ $s_2 \rightarrow 1$,

$\tilde{l} = 1$ bit/símbol (“un 23% més”) .

- Si codifiquem parells de símbols:

- ▶ $s_1 s_1 \rightarrow 010, (1/16)$
- ▶ $s_1 s_2 \rightarrow 011, (3/16)$
- ▶ $s_2 s_1 \rightarrow 00, (3/16)$
- ▶ $s_2 s_2 \rightarrow 1, (9/16)$

$\tilde{l} = 1.688$ bits/2 símbols; 0.844 bits/símbol

- Si codifiquem tríos de símbols:

- ▶ $s_1 s_1 s_1 \rightarrow 11100, (1/64)$
- ▶ $s_1 s_1 s_2 \rightarrow 11101, (3/64)$
- ▶ $s_1 s_2 s_1 \rightarrow 11110, (3/64)$
- ▶ $s_2 s_1 s_1 \rightarrow 11111, (3/64)$
- ▶ $s_2 s_2 s_1 \rightarrow 100, (9/64)$
- ▶ $s_2 s_1 s_2 \rightarrow 1010, (9/64)$
- ▶ $s_1 s_2 s_2 \rightarrow 110, (9/64)$
- ▶ $s_2 s_2 s_2 \rightarrow 0, (27/64)$

$\tilde{l} = 2.469$ bits/3 símbols; 0.823 bits/símbol (“un 1% més”)

En aquest exemple observem que podem disminuir el nombre de bits per símbols si incrementem el nombre de símbols codificats a la vegada. Ens podem preguntar:

- i) Hi ha límit al # bits/símbol?
- ii) Quin és?
- iii) Val la pena?

Definició

Extensió k -èsima de la font $\mathcal{S} = (S, \mathcal{P})$: $\mathcal{S}^k = (S^k, \mathcal{P}^k)$ sent:

$$S^k = S \times S \times \cdots \times S,$$

$$s \in S^k, \quad s = s_{i_1} s_{i_2} \cdots s_{i_k}, \quad s_{i_j} \in S,$$

$$\mathcal{P}^k(s) = \mathcal{P}(s_{i_1}) \mathcal{P}(s_{i_2}) \cdots \mathcal{P}(s_{i_k}).$$

Teorema

Sigui $\mathcal{S}$ una font i $\mathcal{S}^k$ la seva extensió k -èsima, aleshores

$$H(\mathcal{S}^k) = k H(\mathcal{S})$$

Proof.

$$\begin{aligned} H(S^k) &= \sum_{i_1} \sum_{i_2} \cdots \sum_{i_k} p_{i_1} p_{i_2} \cdots p_{i_k} \log \frac{1}{p_{i_1} p_{i_2} \cdots p_{i_k}} \\ &= \sum_{i_1} \cdots \sum_{i_k} p_{i_1} \cdots p_{i_k} \log \frac{1}{p_{i_1}} + \dots + \sum_{i_1} \cdots \sum_{i_k} p_{i_1} \cdots p_{i_k} \log \frac{1}{p_{i_k}} \end{aligned}$$

Notem que:

$$\sum_{i_1} \cdots \sum_{i_k} p_{i_1} \cdots p_{i_k} \log \frac{1}{p_{i_1}} = \left(\sum_{i_1} p_{i_1} \log \frac{1}{p_{i_1}} \right) \left(\sum_{i_2} p_{i_2} \right) \cdots \left(\sum_{i_k} p_{i_k} \right) = H(S) \cdot 1 \cdots 1.$$

Per tant:

$$H(S^k) = H(S) + \cdots + H(S) = k H(S).$$



Observacions:

- i) L'entropia per símbol no canvia $H(\mathcal{S}^k)/k = H(\mathcal{S})$.
- ii) Estem suposant una font sense memòria, o sigui que l'aparició d'un símbol no condiciona la probabilitat d'aparició del següent símbol. No sempre passa, per exemple, si estem llegint un text i apareix q , la propera lletra serà, probablement, u .

Primer teorema de Shannon – codificació sense soroll, V2

Sigui $\mathcal{C}_K$ un codi de Huffman per a l'extensió k -èsima de la font $\mathcal{S}$. Aleshores:

$$\lim_{k \rightarrow \infty} \frac{\tilde{l}(\mathcal{C}_K)}{k} = H(\mathcal{S}).$$

Proof.

$$k H(\mathcal{S}) = H(\mathcal{S}^k) \leq \tilde{l}(\mathcal{C}_K) < H(\mathcal{S}^k) + 1 = k H(\mathcal{S}) + 1$$

Dividint per k :

$$H(\mathcal{S}) \leq \frac{\tilde{l}(\mathcal{C}_K)}{k} < H(\mathcal{S}) + \frac{1}{k}.$$



Exemples de codis de Huffman per llengües

LETTER	#	CODE	LETTER	#	CODE	LETTER	#	CODE
	216026	111	i	65471	0110	r	52173	0001
a	77948	1010	j	1084	1011111010	s	64256	0101
b	16886	100101	k	8058	10111111	t	88037	1101
c	22523	110011	l	42804	11000	u	26707	00001
d	38233	10110	m	23285	00000	v	8606	1011110
e	117141	001	n	65645	0111	w	22227	110010
f	20846	101110	o	69357	1000	x	1034	1011111001
g	20827	100111	p	17265	100110	y	16877	100100
h	62917	0100	q	1556	10111111011	z	632	1011111000

Source: (clean) Moby Dick, by Herman Melville.

1 168 421 characters of an alphabet of 27 letters.

$$\log_2(27) \approx 4.75489$$

Entropy: $H(S) \approx 4.094353$ bits.

Mean length of code $\tilde{\ell}(\mathcal{H}) \approx 4.134070$ bits per character.

Exemples de codis de Huffman per llengües

LETTER	#	CODE	LETTER	#	CODE	LETTER	#	CODE
	381208	00	i	90077	11101	s	125728	1010
a	200499	010	j	10530	11110100	t	61749	10011
b	24147	1011111	l	89143	11100	u	79560	11010
c	59437	10010	m	44658	111100	v	17856	1011011
d	87240	11011	n	112683	1000	w	2	11110101000
e	229191	011	o	162514	1100	x	377	11110101001
f	7581	111101011	p	35465	100110	y	25115	1111011
g	17225	1011010	q	32483	101100	z	6491	1111010101
h	19920	1011110	r	100955	11111			

Source: (clean) Don Quijote de la Mancha, by Miguel de Cervantes.

2021 834 characters of an alphabet of 26 letters.

$$\log_2(26) = 4.70044 \dots$$

Entropy: $H(S) = 3.968888 \dots$ bits.

Mean length of code $\tilde{\ell}(\mathcal{H}) = 4.017647 \dots$ bits per character.

Observacions:

- i) L'entropia marca un límit a la compressió sense pèrdues.
- ii) Ens podem apropar tant com vulguem al límit estenent la font, però la convergència pot ser molt lenta.
- iii) Tot i que matemàticament està clar què és l'entropia un cop definit el model, el problema rau a modelitzar la font. D'aquí l'existència de diferents mètodes de compressió i dels seus resultats diferents.

Codis adaptatius

A l'hora de construir el codi es pressuposen conegudes les freqüències, però no sempre és així. És convenient adaptar les probabilitats a la situació concreta:

- Llegir el fitxer dues vegades, una per calcular les freqüències i una altra per codificar-lo un cop construït el codi. Aquest seria un mètode semiadaptatiu. Cal enviar el codi.
- Anar construint el codi a mesura que es llegeix el fitxer. En aquest cas no és necessari enviar el codi. Permet començar a codificar sense necessitat de llegir tot el missatge i *reinicialitzar* el codi utilitzat a meitat de la lectura/transmissió quan la fdp dels símbols varia considerablement.

Mètode adaptatiu. Inicialment l'arbre només té un node que representa **ESC**:

- Si es llegeix un símbol que no està a l'arbre s'escriu el codi corresponent a **ESC** i el símbol tal qual (ASCII, UTF8,...)
- Si es llegeix un símbol que està a l'arbre s'escriu el codi corresponent al símbol i s'actualitza l'arbre (pàgina següent).

Cada node de l'arbre tindrà etiquetes

$$\boxed{\text{SIM } \omega_i(V_i)}$$

on **SIM** és el símbol (si és fulla) o buit (si node intern); el *pes* ω_i d'una fulla és el nombre de cops que ha aparegut **SIM** fins al moment, pels nodes interns és la suma dels pesos dels fills

Per actualitzar l'arbre de forma eficient se li demana que conservi la *sibling property*:

- Es numeren els vèrtexs de manera que V_{2k-1} i V_{2k} tinguin el mateix pare i que l'índex del pare sigui major que el dels fills, o sigui que el pare sigui V_{2k+1+i} , $i \geq 0$.
- A més demanem que els pesos associats als vèrtexs estiguin ordenats segons la seva numeració $\omega_i \leq \omega_{i+1}$ sent ω_i el pes associat al vèrtex V_i . (Podem suposar que numerem els vèrtexs des de la fulla de més a la dreta fins l'arrel, de dreta a esquerra i per nivells; aleshores els pesos han de quedar ordenats també segons aquest recorregut)

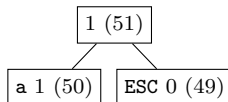
Exemple: aardvark (I)

Text a codificar: **aardvark** (necessitem $2 \cdot 26 - 1$ nodes)*

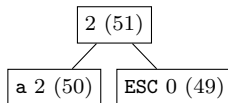
Arbre Inicial:

ESC 0 (51)

- Llegim **a**, no està a l'arbre, enviem el codi ASCII de **a**: $0x61 = 01100001$.
- Missatge codificat: 01100001
- Actualitzem l'arbre:



- Llegim **a**, està a l'arbre, enviem el codi de **a**: 0.
- Missatge codificat: 01100001 0
- Actualitzem l'arbre:

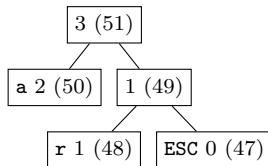


- Llegim **r**, no està a l'arbre, enviem el codi de **ESC**: 1 i el codi ASCII de **r**: $0x72 = 01110010$.
- Missatge codificat: 011000010 1 01110010

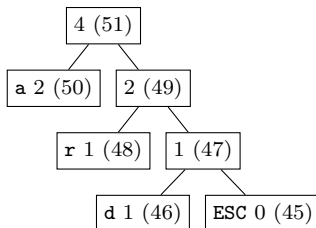
* (Si sabem que els símbols són les 26 lletres)

Exemple: aardvark (II)

- Actualitzem l'arbre:



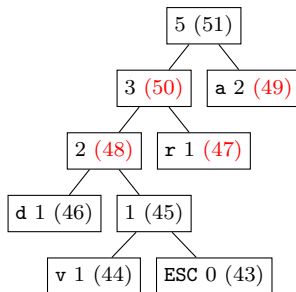
- Llegim **d**, no està a l'arbre, enviem el codi de **ESC**: 11 i enviem el codi ASCII de **d**: $0x64 = 01100100$.
- Missatge codificat: 011000010101110010 11 01100100
- Actualitzem l'arbre:



- Llegim **v**, no està a l'arbre, enviem el codi de **ESC**: 111 i enviem el codi ASCII de **v**: $0x76 = 01110110$.

Exemple: aardvark (III)

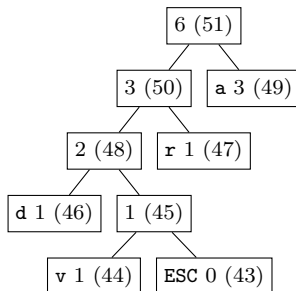
- Missatge codificat: 0110000101011100101101100100 111 01110110
- Actualitzem l'arbre:



- Llegim **a**, està a l'arbre, enviem el codi de **a**: 1.
- Missatge codificat: 011000010101110010110110010011101110110 1

Exemple: aardvark (IV)

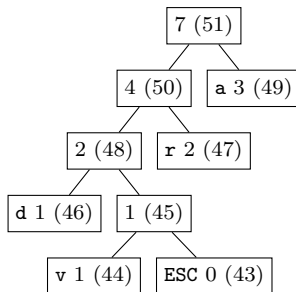
- Actualitzem l'arbre:



- Llegim **r**, està a l'arbre, enviem el codi de **r**: 01.
- Missatge codificat: 0110000101011100101101100100111011101101 01

Exemple: aardvark (V)

- Actualitzem l'arbre:



- ...

Exemple: Rebem 011000010101110010110110010011101110110101.

- Els 8 primers bits
011000010101110010110110010011101110110101 corresponen a a. Missatge a.
Construïm l'arbre 24.
- Llegim 011000010101110010110110010011101110110101 que a l'arbre correspon a a.
Missatge aa. Actualitzem l'arbre 24.
- Llegim 011000010101110010110110010011101110110101 que a l'arbre correspon a ESC,
els 8 següents bits
011000010101110010110110010011101110110101 corresponen a r. Missatge aar.
Actualitzem l'arbre 25.
- Llegim 011000010101110010110110010011101110110101 que a l'arbre correspon a ESC,
els 8 següents bits
011000010101110010110110010011101110110101 corresponen a d. Missatge aard.
Actualitzem l'arbre 25.
- Llegim 011000010101110010110111001011101110110101 que a l'arbre correspon a ESC,
els 8 següents bits
011000010101110010110110010011101110110101 corresponen a v. Missatge aardv.
Actualitzem l'arbre 26.
- Llegim 01100001010111001011011001001110111011010101 que a l'arbre correspon a a.
Missatge aardva. Actualitzem l'arbre 27.
- Llegim 01100001010111001011011001001110111011010101 que a l'arbre correspon a r.
Missatge aardvar. Actualitzem l'arbre 28 ...

Codis de Shannon-Fao

- 1 S'ordenen els símbols segons les seves probabilitats.
- 2 Es divideixen en dos subconjunts de probabilitats semblants, al primer subconjunt se li assigna 0 i al segon 1.

Per a cada subconjunt es procedeix de forma anàloga fins que tots els subconjunts tinguin un únic element.

Exemple: Font amb fdp $\{0.20, 0.15, 0.25, 0.05, 0.1, 0.15, 0.1\}$.

0.25	0	0			00
0.20	0	1			01
0.15	1	0	0		100
0.15	1	0	1		101
0.1	1	1	0		110
0.1	1	1	1	0	1110
0.05	1	1	1	1	1111

$\tilde{l} = 2.7$ bits/símbol, en aquest cas és la mateixa que Huffman (tot i que en molts casos pugui coincidir amb la d'un codi de Huffman no sempre és així).